Habib University
shaping futures

CS201 – Data Structures II

Lecture 06

Dr. Muhammad Mobeen Movania

Overview

- Issues with Linkedlists
- Introduction to Skiplist
- Basic Structure
- Properties
- Implementations
  - SkiplistSSet and its operations
  - SkiplistList and its operations
- Skiplists analysis
- Summary

Issues with LinkedLists

- Linked lists disadvantage:
  - They require sequential scanning to locate a searched-for element.
  - The search starts from the beginning of the list and stops when either a searched-for element is found or the end of the list is reached without finding this element.

- Ordering elements on the list can speed up searching, but a sequential search is still required.

- Key Idea
  - Think about lists that may allow skipping certain nodes to avoid sequential processing.

Skiplist

- Provides a list implementation of get, set, add, remove functions with a **O(log n)** average complexity and all SSet operations in **O(log n) expected time**

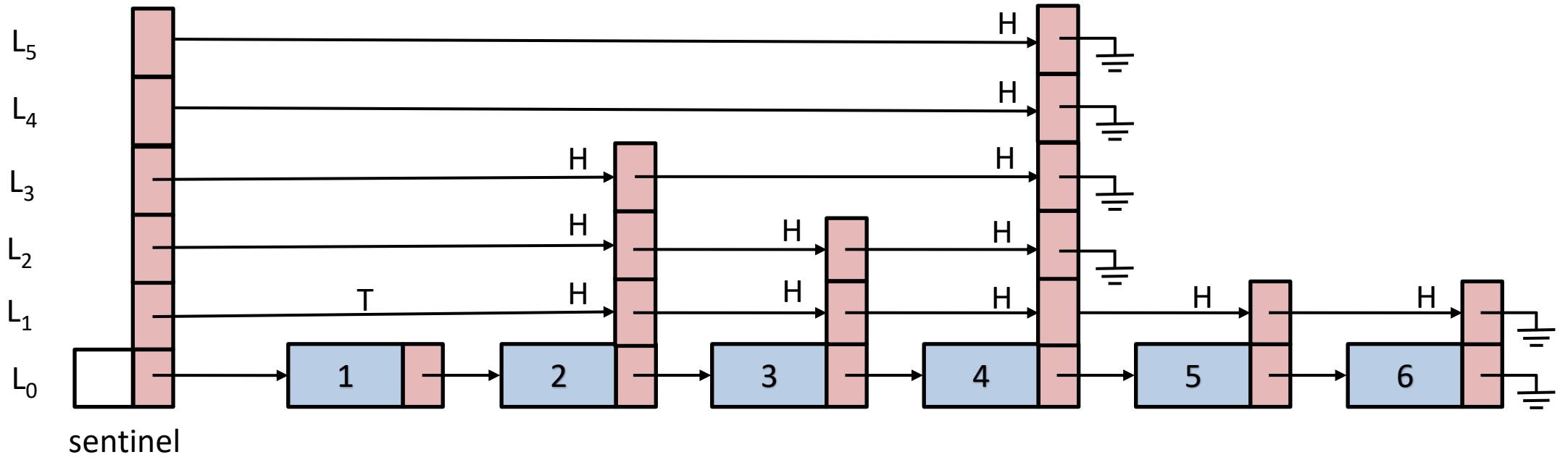
- Efficiency is achieved through assignment of **random height** to the new element added.
  - It uses random coin tosses to determine the height of the new element.

Skiplist – Basic structure

- Skiplist is a sequence of singly-linked lists $L_0, \ldots, L_h$.
- Each L_r contains subset of items in L_{r-1}
- Starting with input list L_0 containing n ordered elements
  - L_1 is constructed using L_0, L_2 from L_1, L_3 from L_2 and so on
- For each element to be added in L_r, a coin is tossed
  - if it turns as heads, the item is added to L_r
  - We continue until L_r is empty

Skiplist generation - Example

- Suppose we have the following data array [1, 2, 3, 4, 5, 6]



Skiplist - Properties

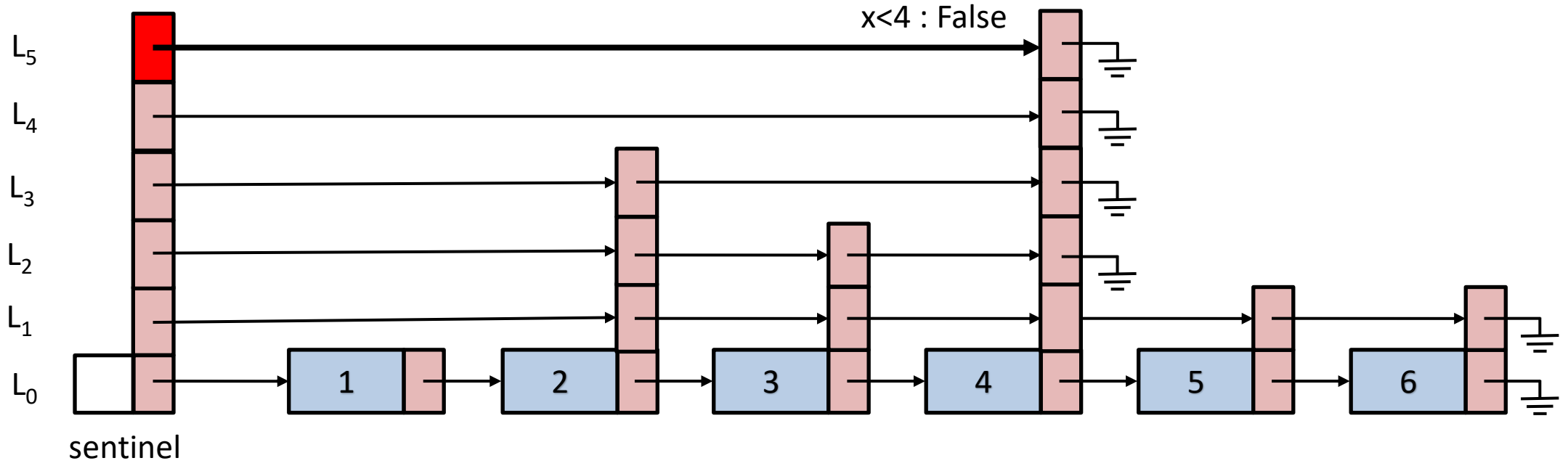
- Height of an element *x*
  - The largest value *r* such that *x* appears in L_r
  - Intuitively the height is the total number of heads that came during the random dice of coin
- Height of a skiplist is the height of its tallest node
- Examples from previous skiplist:
  - Height of 1 is 0
  - Height of 5,6 is 1
  - Height of 3 is 2
  - Height of 2 is 3
  - Height of 4 is 5
  - Tallest node is 4 with a height of 5
- Height of previous skiplist is 5

Skiplist – Properties (2)

- Sentinel
  - A special node that keeps track of the shortest path (**the search path**) from sentinel in L_h to every node in L_0.
- To search for an element *u* in skiplist
  - Start from top left corner of skiplist at level L_h and go right until we have an element which appears before u in L_0
    - If there exists an element, we move right using the next pointer at the current level L_h
    - Otherwise we step down to lower level L_{h-1}
  - Continue doing the above steps until we have the predecessor of u in L_0
- Expected length of search path from any node in L_0 is O(log n)
- Two implementations discussed: SkiplistSSet and SkiplistList

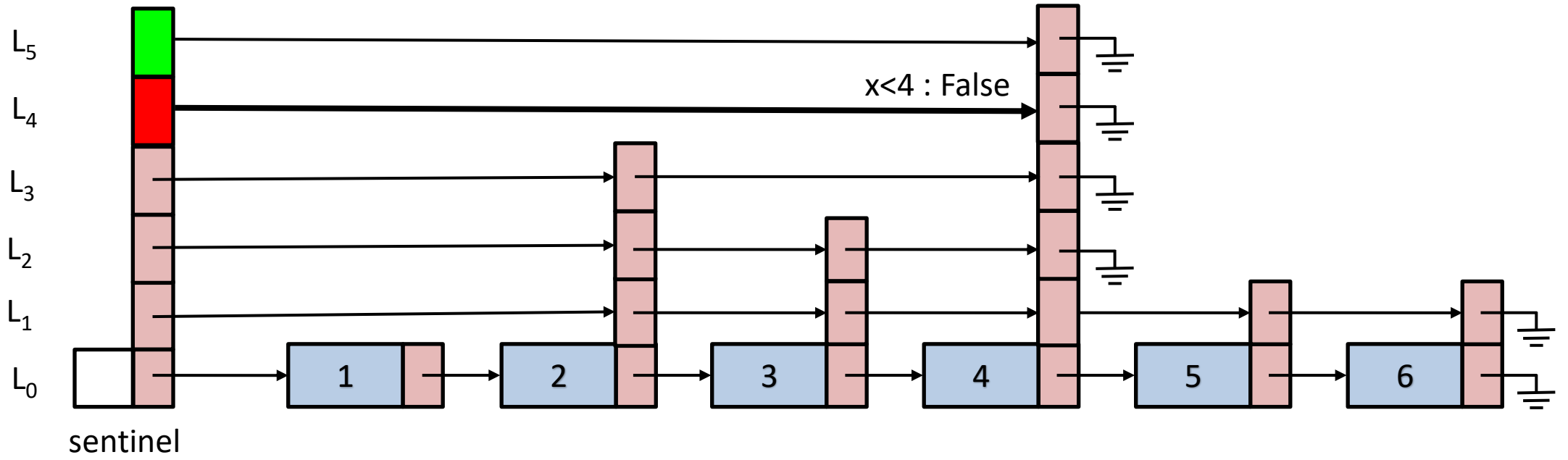
Skiplist Search Path - Example

- Suppose we have the following data array [1, 2, 3, 4, 5, 6]
- We want to find the path to node with value 4



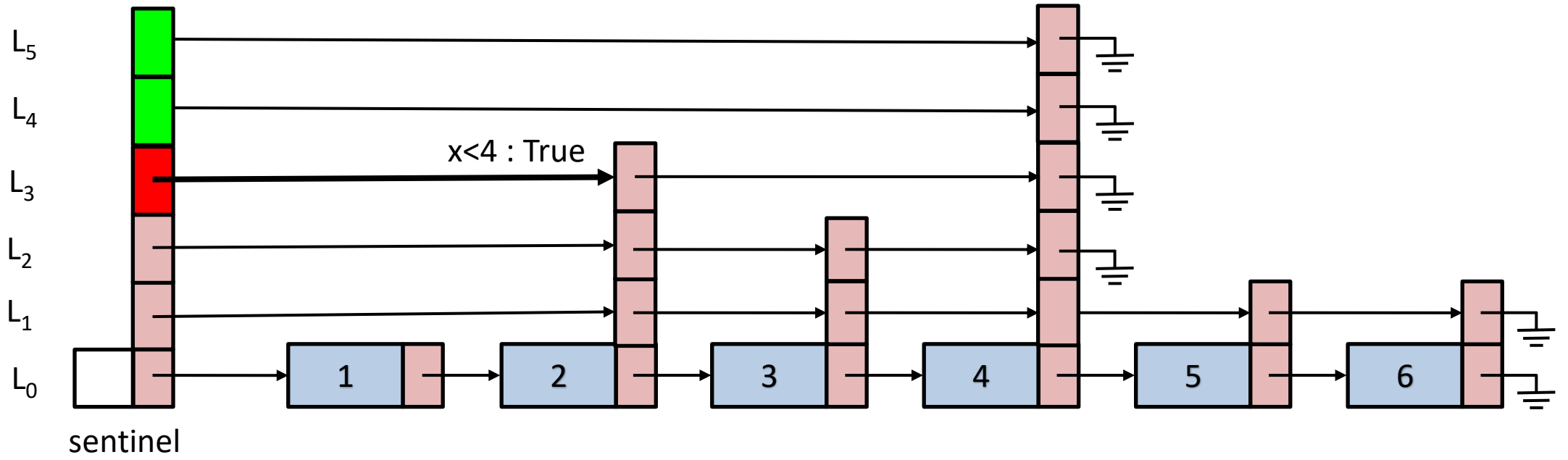
Skiplist Search Path - Example

- Suppose we have the following data array [1, 2, 3, 4, 5, 6]
- We want to find the path to node with value 4



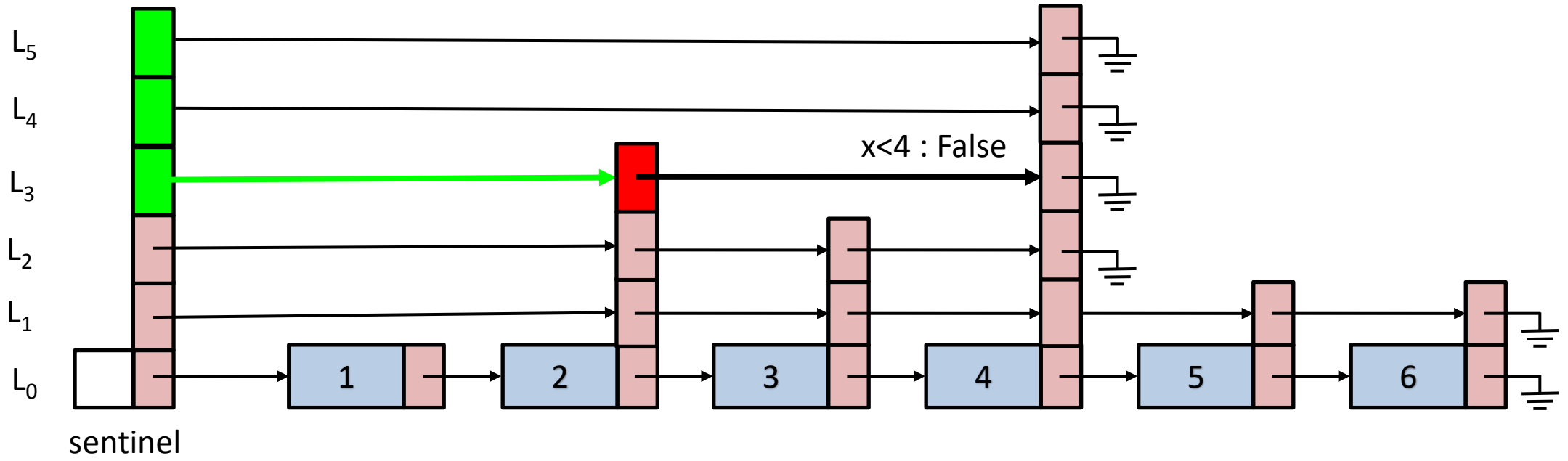
Skiplist Search Path - Example

- Suppose we have the following data array [1, 2, 3, 4, 5, 6]
- We want to find the path to node with value 4



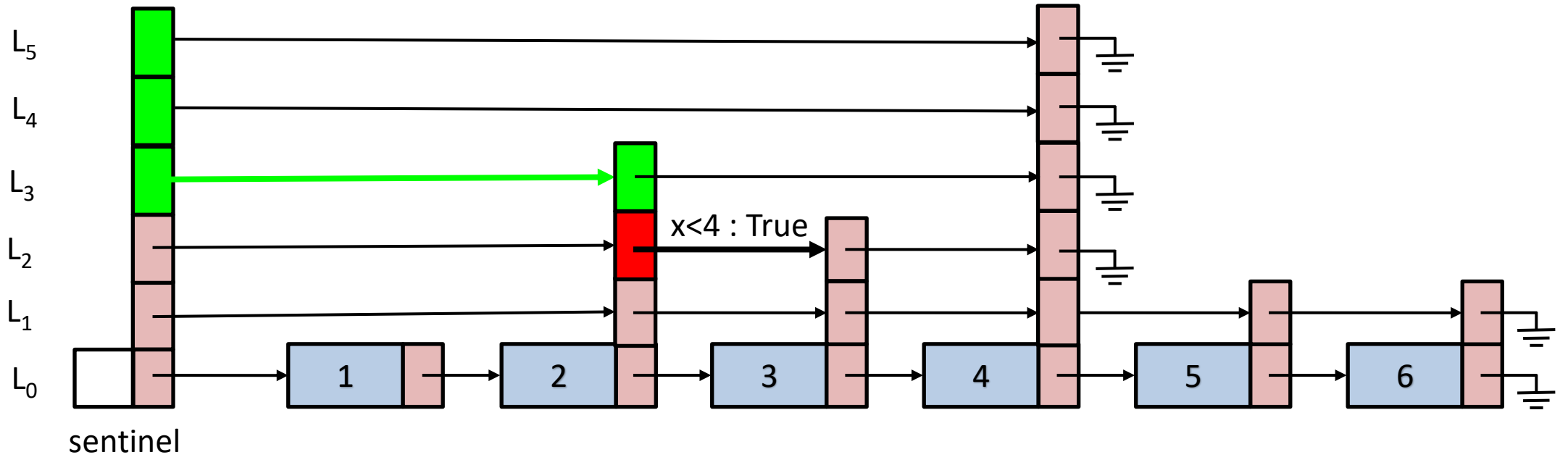
Skiplist Search Path - Example

- Suppose we have the following data array [1, 2, 3, 4, 5, 6]
- We want to find the path to node with value 4



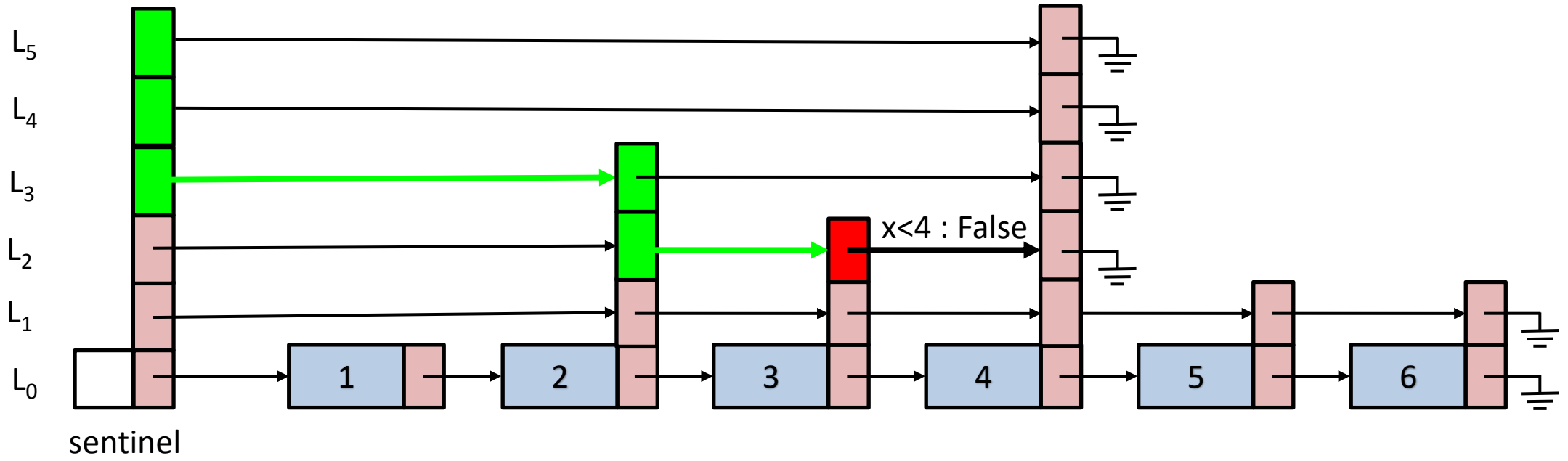
Skiplist Search Path - Example

- Suppose we have the following data array [1, 2, 3, 4, 5, 6]
- We want to find the path to node with value 4



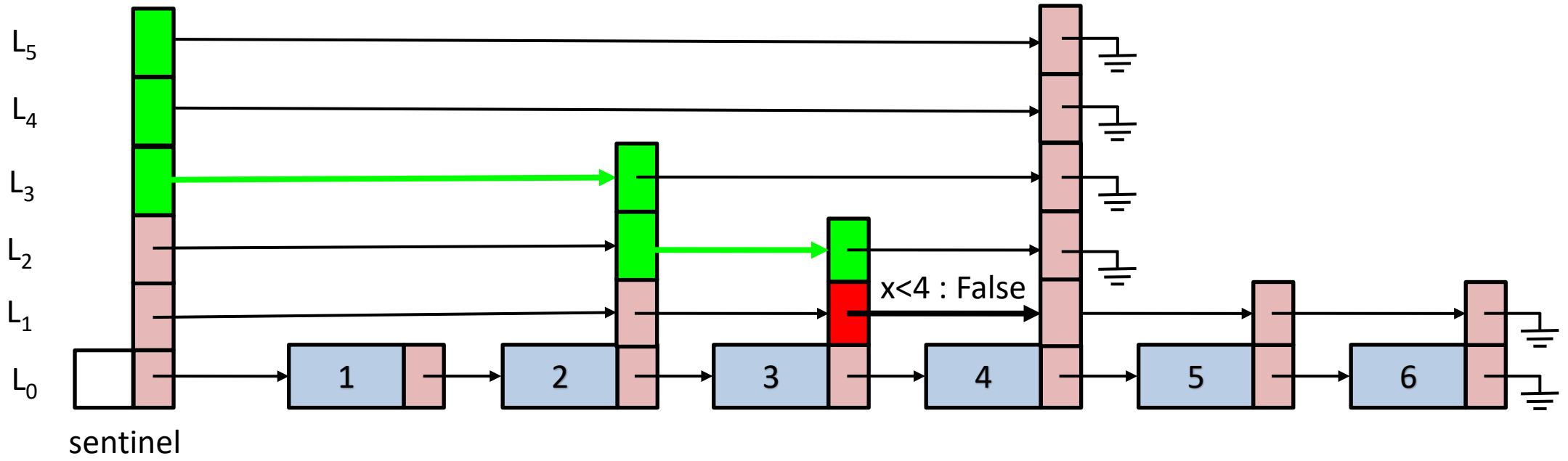
Skiplist Search Path - Example

- Suppose we have the following data array [1, 2, 3, 4, 5, 6]
- We want to find the path to node with value 4



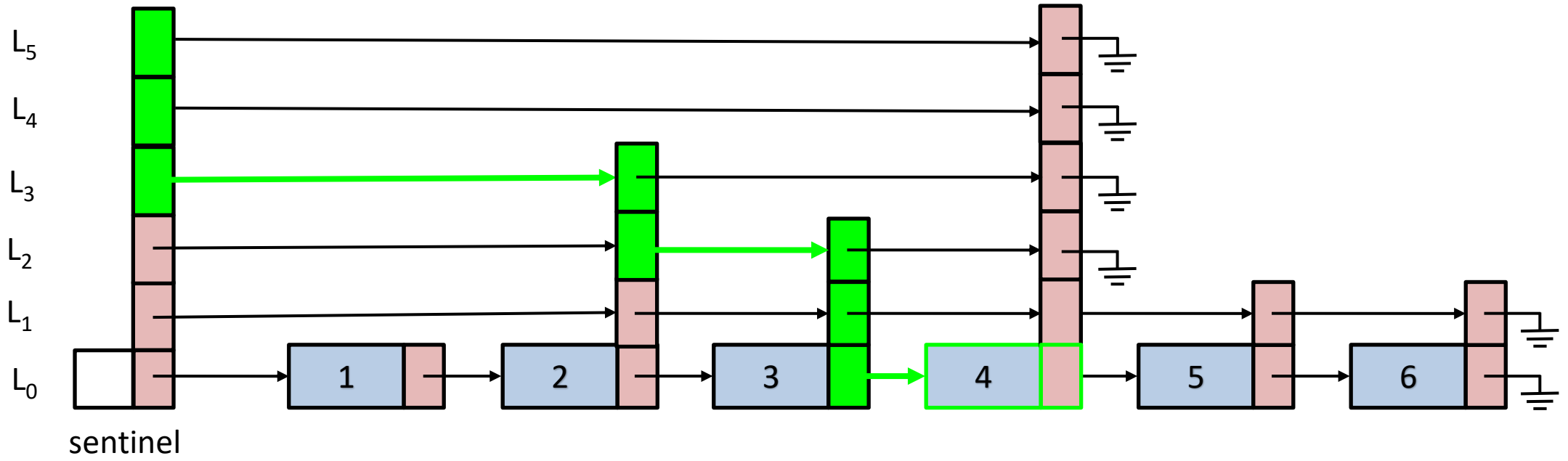
Skiplist Search Path - Example

- Suppose we have the following data array [1, 2, 3, 4, 5, 6]
- We want to find the path to node with value 4



Skiplist Search Path - Example

- Suppose we have the following data array [1, 2, 3, 4, 5, 6]
- We want to find the path to node with value 4



Skiplists – Implementation details


```
template<class T>
class SkiplistList
{
   protected:
      T null;                  //used for returning null value during comparison
      struct Node
      {
         T x;                  //data value
         int height;           //height of current node
         int *length;          //edge lengths for all nodes in path
         Node **next;          //list of pointers pointing to successors of
                               //current node
      };
      Node *sentinel;          //sentinel node
      int h;                   //height of the skiplist
      int n;
      //rest of the implementation of SkiplistList
};
```

SkiplistSSet: An Efficient SSet Implementation

- List L_0 stores elements of SSet in **sorted order**
- find(x) follows the search path for the smallest value y such that y >= x
- Expected running time of find: O(log n)

SkiplistSSet::find function


```
Node* SkiplistSSet::findPredNode(T x)
{
    Node *u = sentinel;
    int r = h;                  //used to go down from level h to h-1
    while (r >= 0) {
        while (u->next[r] != NULL && compare(u->next[r]->x, x) < 0)
            u = u->next[r];     // go right in list r
        r--;                    // go down into list r-1
    }
    return u;
}

T SkiplistSSet::find(T x) {
    Node *u = findPredNode(x);
    return u->next[0] == NULL ? null : u->next[0]->x;
}
```

SkiplistSSet::pickHeight function

- Simulates the flipping of coin by counting the number of trailing 1s in a random num.

```
int SkiplistSSet::pickHeight()
{
    int z = rand();
    int k = 0;
    int m = 1;
    while ((z & m) != 0)
    {
        k++;
        m <<= 1;
    }
    return k;
}
```

SkiplistSSet::add function

- Proceeds in 5 steps
  - We first try to find the predecessor node of the node containing x so we can insert it at L_0
  - We splice x into few lists: $L_0 \ldots L_k$ (k is obtained through pickHeight)
  - All nodes on the search path are tracked using an array called stack.
    - stack[r] is the node in L_r where search proceeded down to L_{r-1}
    - stack[0]..stack[k] are the nodes modified when x is inserted
  - The height k is checked against the current height of skiplist to ensure that the sentinel is stored in case if the current height given by pickHeight is greater.
  - Finally, the tracked search path stored in the array (stack) is assigned to the newly added node

- Expected runtime: O(log n)

SkiplistSSet::add function

```
bool add(T x) {
    Node *u = sentinel;
    int r = h;
    int comp = 0;
    while(r >= 0)
    {
        while(u->next[r] != NULL &&
              (comp = compare(u->next[r]->x, x)) < 0)
            u = u->next[r];
        if (u->next[r] != NULL && comp == 0)
            return false;
        stack[r--] = u; // going down, store u
    }
```

Finding predecessor node

Storing search path nodes

SkiplistSSet::add function

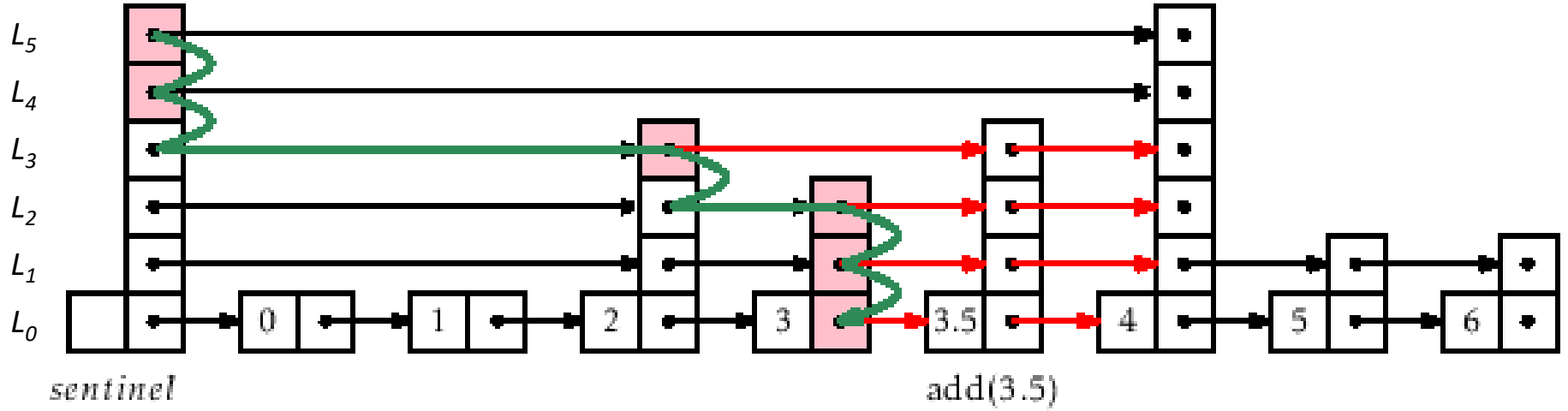
```
    Node *w = newNode(x, pickHeight());
    while (h < w->height)
        stack[++h] = sentinel; // height increased
    for (int i = 0; i <= w->height; i++) {
        w->next[i] = stack[i]->next[i];
        stack[i]->next[i] = w;
    }
    n++;
    return true;
}
```

Ensuring that the we store sentinel in case the pickHeight returns a greater height. This is to prevent null reference

We assign the search path stored in array stack to the new node.

Example of add

- Find the predecessor of 3.5 which comes out as 3
- Next, height is obtained using a flip of coin using pickHeight
- Finally, The search path is added to the new node

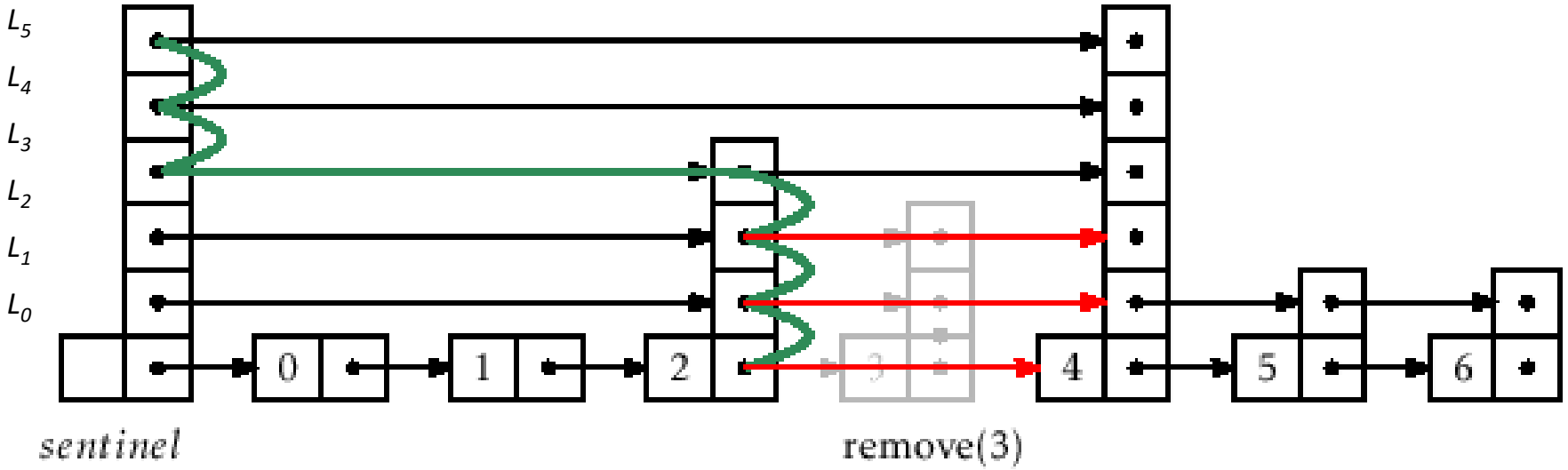


Search path for add(3.5)

SkiplistSSet::remove function

- Removal does not need tracking of search path as the node is deleted during traversal
- We start with a search for the node x at the skiplist height h
- Each time the search moves downward from a node u, we check if u.next.x = x and if so, we splice u out of the list
- Code is very similar to add function so is not discussed here.
- Expected runtime: O(log n)

Example of remove

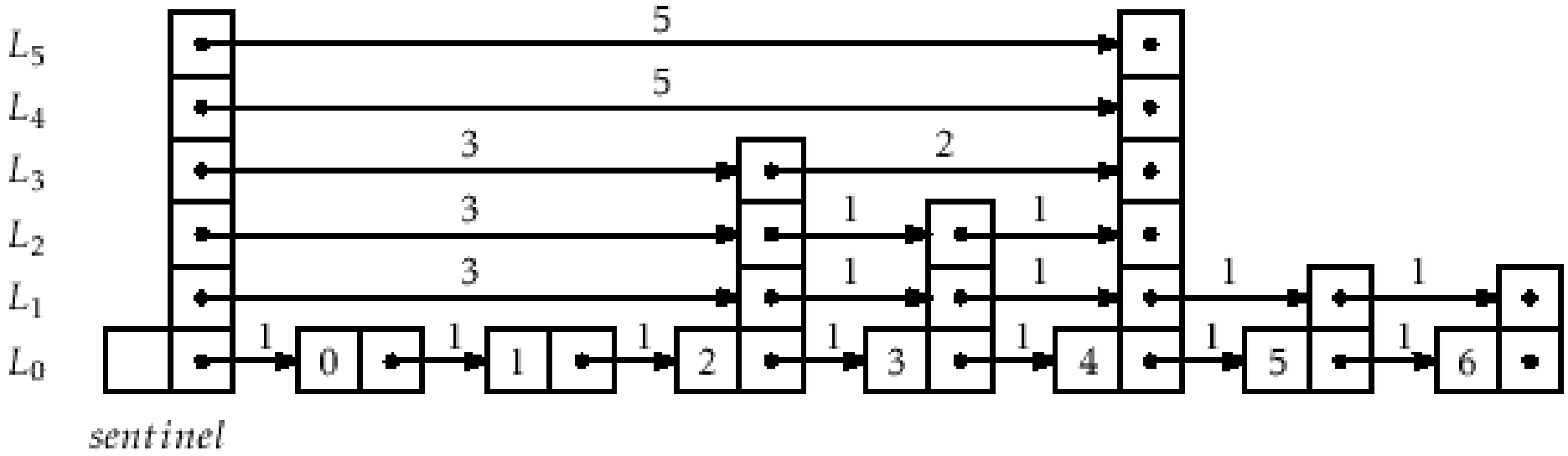


Search path for remove(3)

SkiplistList: An efficient random access list

- Implements the List interface using skiplist structure
- L_0 contains elements of list in the same order as they appear in the list
- Elements can be added, removed, accessed in O(log n) time
- Need a way to follow the search path for the i^{th} element in L_0
  - The easiest way to do this is to define the notion of **length of an edge** in some list, L_r .
- Length of each edge at L_0 is 1, for all following levels, edge length is the sum of all edges below

Edge lengths



Why use edge lengths?

- If we are currently at a node that is at position j in L_0 and we follow an edge of length l, then we move to a node whose position, in L_0, is j + l.

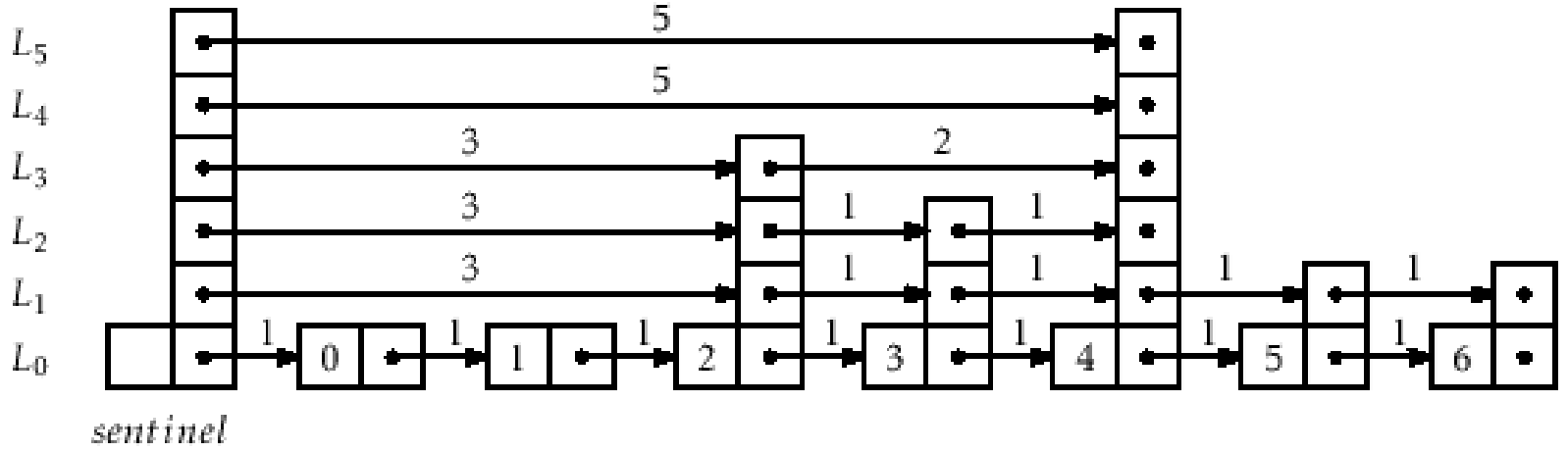
- Advantage
  - While following a search path, we can keep track of the position, j, of the current node in L_0.
  - Can also help determine where to go next
    - When at a node, u, in L_r , we go right if j plus the length of the edge u.next[r] is less than i. Otherwise, we go down into L_{r-1}.

- Whenever the search path goes down at a node, u, in L_r, the length of the edge u.next[r] increases by one, since we are adding an element below that edge at position i.

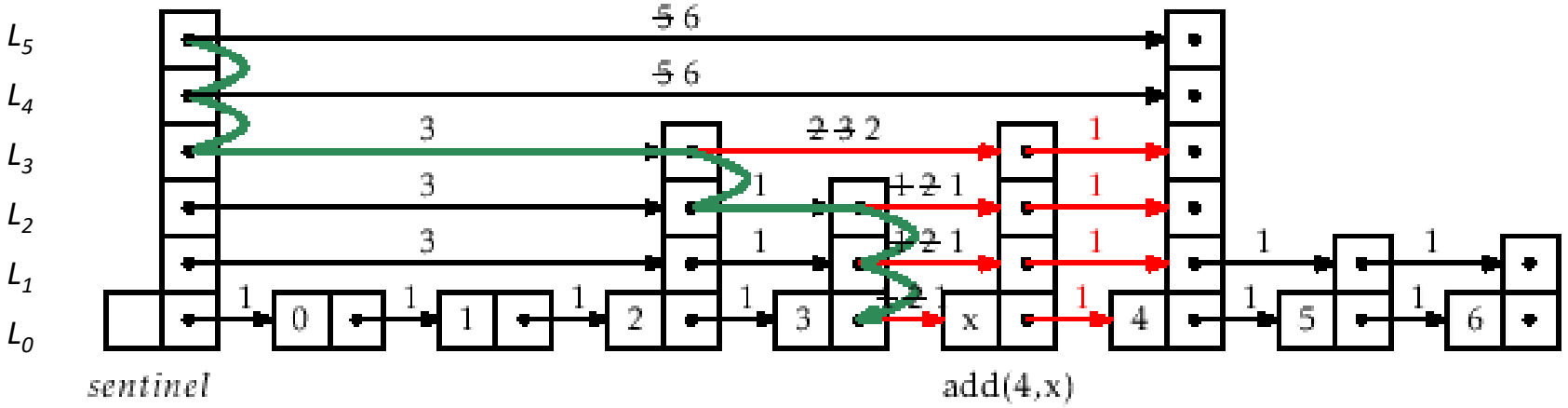
SkiplistList::add function

```
Node* add(int i, Node *w)
{
  Node *u = sentinel;   int k = w->height;   int r = h;
  int j = -1;                                          // index of u
  while (r >= 0)
  {
     while (u->next[r] != NULL && j + u->length[r] < i)
     {
        j += u->length[r];                             //update length of current edge
        u = u->next[r];                                //move to next node at the current level
     }
     u->length[r]++;                                   // to account for new node in list 0
     if (r <= k)
     {
        w->next[r] = u->next[r];                       //ensure that the lengths and the links
        u->next[r] = w;                                //are updated
        w->length[r] = u->length[r] - (i - j);
        u->length[r] = i - j;
     }
     r--;                                              //move one level down
   }
   n++;                                                //increase the total count of nodes
   return u;
}
```

SkiplistList add example



Before add(4, x) call



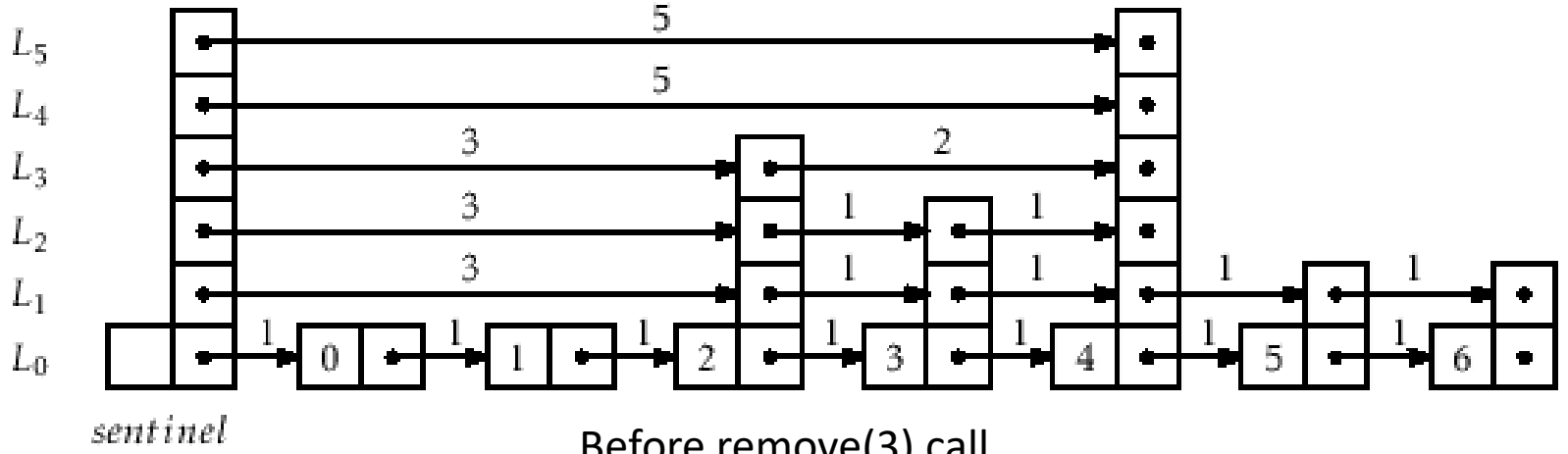
After add(4, x) call

SkiplistList::remove function

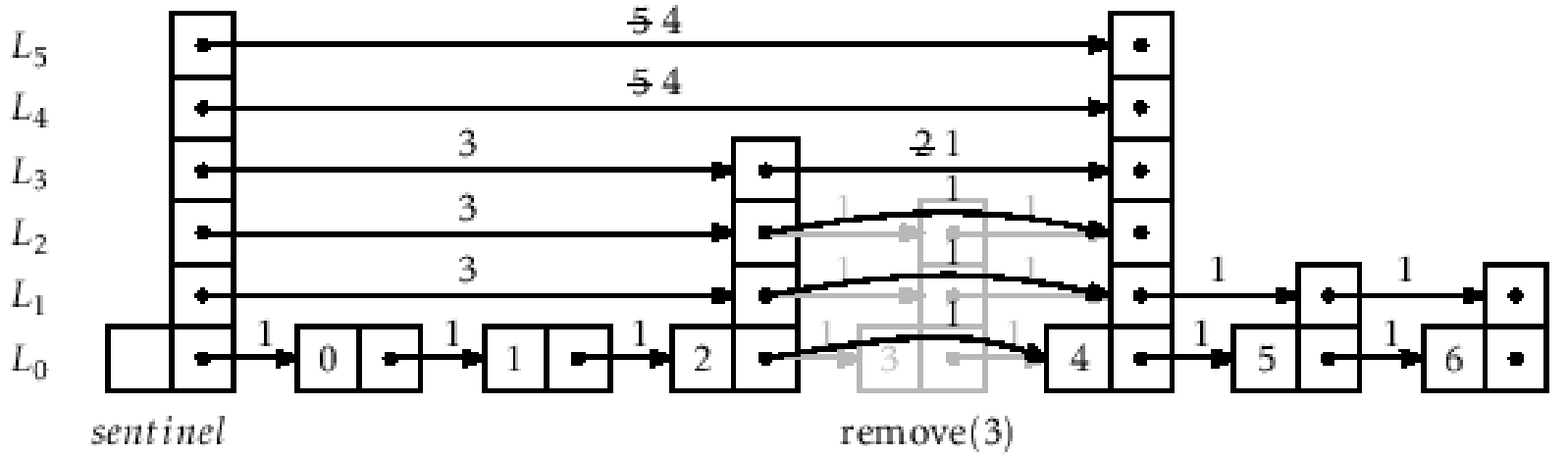
- Three basic steps
  - Follow the search path for the node at position i.
  - Each time the search path takes a step down from a node, u, at level r, decrement the length of the edge leaving u at that level.
  - if u.next[r] is the element of rank i, if so, splice it out of the list at that level.

- The code for remove is similar to add so it is not produced here.

SkiplistList remove example



Before remove(3) call



After remove(3) call

Skiplist Analysis

- **Lemma 4.2.:** Let T be the number of times a fair coin is tossed up to and including the first time the coin comes up heads. Then E[T] = 2.
- Proof:
  - We stop tossing the coin when we first get a head
  - We define an indicator variable I_i which is set to 0 (if coin is tossed <i times) and 1 otherwise. This means that we had tails in i-1 previous coin tosses

$$I_i = \begin{cases} 0 \\ 1 \end{cases}$$

$$E(I_i) = \Pr(I_i = 1) = \frac{1}{2^{i-1}}$$

Total coin tosses T = $\sum_{i=1}^{\infty} I_i$

Skiplist Analysis – Lemma 4.2 proof

$$
\begin{aligned}
\mathrm{E}[T] &= \mathrm{E}\left[\sum_{i=1}^{\infty} I_i\right] \\
&= \sum_{i=1}^{\infty} \mathrm{E}[I_i] \\
&= \sum_{i=1}^{\infty} 1/2^{i-1} \\
&= 1 + 1/2 + 1/4 + 1/8 + \cdots \\
&= 2 \ .
\end{aligned}
$$

Skiplist Analysis

- **Lemma 4.3.:** The expected number of nodes in a skiplist containing n elements, not including occurrences of the sentinel, is 2n
- Proof:
  - A node is skiplist is selected at level L_k depends on the outcome of a fair coin toss coming as head
  - Probability of selection a node at L_0 is $p_0=1$ as all nodes are always included in L_0
  - For level L_1 , $p_1=1/2$ (subjected to the fair coin toss being head)
  - For level L_2, $p_2=1/2*p_1=1/2^2$ since the nodes from L_1 will be used at level L_2
  - For level L_r, $p_r=1/2^r$
- Expected number of nodes at level $L_r=n/2^r$
- Expected number of nodes at all levels = $\sum_{i=0}^{\infty} n/2^i = n\left(\sum_{i=0}^{\infty} \frac{1}{2^i}\right) = 2\text{n}$

Skiplist Analysis

- **Lemma 4.4.:** The expected height of a skiplist containing n elements is at most logn+2
- Proof:
  - Let r be the level of skiplist: r={1,2,3,…,∞}
  - We define an indicator random variable: $I_r = \begin{cases} 0 & if\ L_r\ is\ empty \\ 1 & if\ L_r\ is\ not\ empty \end{cases}$
  - Height of the skiplist: $h = \sum_{r=0}^{\infty} I_r$
  - I_r is never more than the length, $|L_r|$, of L_r:

$$E[I_r] \leq E[|L_r|] = n/2^r$$

Skiplist Analysis – Lemma 4.4 proof

$$
\begin{aligned}
\mathrm{E}[\mathrm{h}] &= \mathrm{E}\left[\sum_{r=1}^{\infty} I_r\right] \\
&= \sum_{r=1}^{\infty} E[I_r] \\
&= \sum_{r=1}^{\lfloor \log n \rfloor} E[I_r] + \sum_{r=\lfloor \log n \rfloor + 1}^{\infty} E[I_r] \\
&\leq \sum_{r=1}^{\lfloor \log n \rfloor} 1 + \sum_{r=\lfloor \log n \rfloor + 1}^{\infty} n/2^r \\
&\leq \log n + \sum_{r=0}^{\infty} 1/2^r \\
&= \log n + 2 \ .
\end{aligned}
$$

The value of I_r cannot be great than 1 for all levels less than or equal to log n , and for the rest, $E[I_r]$ can never be more than total number of nodes at level r i.e. $n/2^r$:

Skiplist Analysis

- **Lemma 4.5.:** The expected number of nodes in a skiplist containing n elements, including all occurrences of the sentinel, is 2n + O(logn).
- Proof:
  - The expected no. of nodes in a skiplist is 2n (see lemma 4.3)
  - There are always h sentinels in a skiplist of height h.
  - The expected height h of skiplist is log(n)+2 (see lemma 4.4)
  - So expected number of nodes in a skiplist with n elements and all occurences of sentinel will be 2n+log(n)+2 = 2n+O(log(n))

Skiplist Analysis

- **Lemma 4.6.:** The expected length of the search path for any node, u, in L_0 is at most 2logn + O(1) = O(logn).
- Proof:
  - S is the search path length for node u in a skiplist with height h
  - Let S_r be the number of forward search path steps at level L_r
  - $E[S_r] <= 1$ and $S_r <= |L_r|$ as we cant take more steps than $|L_r|$

$$E[S_r] \leq E[|L_r|] = n/2^r$$

Skiplist Analysis – Lemma 4.6 proof

$$
\begin{aligned}
E[S] &= E\left[h + \sum_{r=0}^{\infty} S_r\right] \\
&= E[h] + \sum_{r=0}^{\infty} E[S_r] \\
&= E[h] + \sum_{r=0}^{\lfloor \log n \rfloor} E[S_r] + \sum_{r=\lfloor \log n \rfloor + 1}^{\infty} E[S_r] \\
&\leq E[h] + \sum_{r=0}^{\lfloor \log n \rfloor} 1 + \sum_{r=\lfloor \log n \rfloor + 1}^{\infty} n/2^r \\
&\leq E[h] + \sum_{r=0}^{\lfloor \log n \rfloor} 1 + \sum_{r=0}^{\infty} 1/2^r \\
&\leq E[h] + \sum_{r=0}^{\lfloor \log n \rfloor} 1 + \sum_{r=0}^{\infty} 1/2^r \\
&\leq E[h] + \log n + 3 \\
&\leq 2 \log n + 5 = O(\log(n))
\end{aligned}
$$

Summary

- We discussed Skiplist and the two implementation SkiplistSSet and SkiplistList
- We looked at their properties, structures and operations
- We looked at the complexity analysis of the Skiplist operations

Book Readings

- ODS: Chapter 4 pages: 87-102

Next Lecture

- Lecture 07